

利用形状保持采样的任意形状聚类方法

周 泉¹, 薛 亮², 黄 浩¹, 应 时¹

¹ (武汉大学 计算机学院, 武汉 430072)

² (苏州城市学院 计算科学与人工智能学院, 苏州 215104)

E-mail: haohuang@whu.edu.cn

摘 要: 任意形状聚类旨在快速、准确地分析识别出数据集中具有任意分布形状的数据簇。大多数现有方法具有二次方以上的时间复杂度。为降低计算代价, 部分方法通过下采样来减少待分析的数据量, 但采样结果常不能保持原始的数据分布形状, 进而影响最终聚类结果。本文提出一种利用形状保持采样的任意形状聚类方法。首先, 在原始数据集上进行下采样, 使采样点均匀连续地分布于原始的数据分布内, 从而保持原始的数据分布形状; 然后, 基于采样点选取代表点并调整各代表点位置, 使其与近邻的同质代表点靠拢, 提升邻近代表点属于同一聚类的概率; 最后, 基于类间最小距离对代表点进行凝聚聚类, 将邻近代表点逐渐合并到同一聚类中。实验结果表明, 本文方法在聚类结果准确性、运行效率上均优于现有代表性方法。

关键词: 任意形状聚类; 形状保持采样; 凝聚聚类

中图分类号: TP301

文献标识码: A

Arbitrary Shaped Clustering Using Shape Preserving Sampling

ZHOU Quan¹, XUE Liang², HUANG Hao¹, YING Shi¹

¹ (School of Computer Science, Wuhan University, Wuhan 430072, China)

² (College of Computing Science and Artificial Intelligence, Suzhou City University, Suzhou 215104, China)

Abstract: Arbitrary shaped clustering aims to identify data clusters with arbitrary distribution shapes in a dataset efficiently and effectively. Most existing methods suffer from a time complexity of at least quadratic order. To reduce computational costs, some approaches downsample the dataset to decrease the number of data points to be analyzed. Nonetheless, the sampled data often fail to preserve the original distribution shapes, which will impact the final clustering results. This paper proposes an arbitrary shaped clustering method using shape preserving sampling. First, downsampling is performed on the original dataset to ensure that the sampled data points are evenly and continuously distributed within the original data distribution, thereby preserving its shape. Next, select representative data points based on sampled data points, and the positions of the representative data points are adjusted to move closer to their homogeneous neighbors, increasing the likelihood that neighbors belong to the same cluster. Finally, agglomerative clustering is applied with the minimum inter-cluster distance, gradually merging neighboring representative data points into the same cluster. Experimental results demonstrate that the proposed method outperforms existing representative approaches in both clustering accuracy and computational efficiency.

Keywords: arbitrary shaped clustering; shape preserving sampling; agglomerative clustering

0 引 言

聚类是数据挖掘领域重要的开放性研究课题。其核心工作是定义量化标准来衡量数据样本之间的相似性, 使得同一聚类中的数据样本相似度高, 而不同聚类间的数据样本相似度低, 并基于这种相似性设计聚类操作, 将数据集中属于同一聚类的数据样本聚在同一个子集中。

在许多实际应用场景中, 数据样本的分布形状经常不规则, 形成了具有任意分布形状的数据簇。任意形状聚类旨在

能快速、准确分析识别出这些任意形状的数据簇。例如, 共享电动车停放地点等数据的数据分布通常不呈规则形状, 而是具有随机性, 呈不太规则的任意形状。任意形状聚类能帮助找出共享电动车使用的热点区域, 从而指导共享电动车投放和充电桩布置的地点和数量^[1]。

目前, 常用的聚类方法包括划分方法、层次聚类方法、基于图的方法、基于密度的方法等。其中划分方法主要用于发现具有凸的分布形状的聚类, 其余方法常被认为具备一定

的任意形状聚类能力,但往往具有二次方或更高的时间复杂度。为降低计算代价,一些方法尝试通过对原始数据集进行下采样来减少需要分析处理的数据样本数量^[2-7]。所选取的采样策略包括随机采样、基于聚类的采样、基于网格划分的采样等。但是这些采样策略大多存在丢失原始数据簇的分布形状的风险。例如,随机采样往往倾向于更多地出现在数据样本密度较高的区域,而忽视数据样本密度较低的区域;基于聚类的采样可能受到局部密度变化的影响而使得采样结果产生一定程度的形状扭曲或失真;基于网格划分的采样不适合处理不同聚类的分布范围差异较大的情况。这些风险和缺点可能导致现有的基于采样的方法较难保持原始数据簇的分布形状,从而影响任意形状聚类的最终质量。

为了更加快速、准确地分析识别出数据集中的具有任意分布形状的数据簇,本文提出利用形状保持采样的任意形状聚类方法 CAPSULE(由 Arbitrary Shaped Clustering Using shapE Preserving sampLing 中大写粗体字母组合而成)。该方法主要包括以下三个步骤:(1)在原始数据样本分布区域内进行下采样,使采样点均匀连续地分布在原始数据样本分布区域内,从而保持原始数据簇的分布形状;(2)基于采样点选取代表点,并调整各代表点的位置,使其与近邻的同质代表点靠拢,提升邻近代表点属于同一聚类的概率,从而使聚类结构更加清晰,更容易被识别;(3)用类间最小距离对代表点进行凝聚聚类,将邻近代表点逐渐合并到同一聚类中。在人工与真实数据集上的实验结果表明,本文方法在聚类结果准确性、运行效率上均优于现有代表性方法。

1 相关工作

任意形状聚类的相关工作包括:层次聚类方法、基于图的方法、基于密度的方法以及基于采样的方法。

1.1 层次聚类方法

层次聚类方法分为自下而上的凝聚聚类和自顶而下的分裂聚类。凝聚聚类开始时将每个数据样本作为一个单独的类,计算每两个类之间的相似度,合并相似度最大的两个类,重复计算相似度和合并相似度最大两个类的过程,直到类的数量达到指定个数。分裂聚类开始时将所有数据样本归为一类,然后逐步划分成多个类,直到达到停止条件。通常层次聚类方法的时间复杂度为 $O(N^3)$,这里 N 为数据样本的个数。

1.2 基于图的方法

基于图的方法通过将数据样本作为顶点进行构图,将聚类问题转化为图中的子图划分问题。例如,CHAMELEON 算法^[8]是一种典型的基于图的聚类方法。该方法将每个数据样本与其 k 近邻相连构造 k 近邻图,使用顶点之间互连性和邻近性评估子图之间的相似度,然后将相似度高的子图合并来完成聚类。类似方法还包括 SNN^[9]以及 CHKNN^[10]等,这些方法采用了不同的构图方法,例如 SNN 算法使用共享最近邻来衡量相似度并构造图模型,CHKNN 算法使用数据样本间的互 k 近邻和 k 近邻关系共同来构建关系强度图。

谱聚类^[11]并不直接在原始图上划分,而是通过计算所构造图的拉普拉斯矩阵的特征向量来提取图的结构特征,使数据样本在新的表示空间的关系会变得更加线性,从而简化聚类任务。为了提升在复杂数据集上的聚类效果,谱聚类算法还会对邻接矩阵进行优化,例如 MDSC^[12]通过引入高斯混合模型的概率密度提出密度分层的概念,以适应更复杂的多密度数据。通常基于图的方法的时间复杂度一般不低于 $O(N^2)$ 。

1.3 基于密度的方法

基于密度的方法^[13]认为互相靠近的较高密度的数据样本形成了聚类。DBSCAN^[14]及其改进版本如 OPTICS^[15]等是最为常见的基于密度的方法,它们通过找到局部密度超过一定阈值的连通区域来完成聚类操作。另外一种思路是找到局部密度最高或较高的数据样本,然后将周边数据样本向这些高密度数据样本移动(如 DENCLUE^[16]),或者将周边数据样本划归这些高密度数据样本所在的聚类(如 DPC^[17],及其改进版本例如 CLUB^[18],EC^[19],SKM-DPC^[20])。VDPC(变分密度峰值聚类)算法^[21]基于 DPC 的局部密度分析确定数据集的密度分布以及密度级别,然后 VDPC 在各密度级别上以分治的方式确定最终的聚类中心,在每个密度级别中,通过应用 DBSCAN 算法高效且直接地识别所有的聚类。通常基于密度的方法的时间复杂度一般不低于 $O(N^2)$ 。

1.4 基于采样的方法

基于采样的方法通过采样选取代表点来减少待处理的数据样本数量,从而降低计算代价。CURE 算法^[2]是一种经典的基于采样的方法,它通过随机采样的策略获得代表点,然后将代表点划分为一个个小的聚类,继而不断合并这些小聚类得到最终聚类结果。IK-means(Improved K-means)算法^[3]在高密度区域获取较多的子簇,再通过子簇在超平面上的数据密度和两个子簇之间的最近距离来评估子簇之间的连通性,从而合并小聚类。由于随机采样结果常常造成代表点分布不规则,部分算法如 SPARCL^[4]和 ABACUS^[5]等采取了基于聚类的采样策略,使得所选取的代表点更靠近原始数据样本分布的局部密度中心,从而增加代表点的代表性。然而当局部密度变化较大时,这类方法所选的代表点分布形状可能会产生一定的扭曲,从而一定程度地增加聚类识别的难度。为降低代表点的形状扭曲,CLASP 算法^[6]采取了基于网格划分的采样策略,将数据集划分为等大小的网格,并从数据样本数量超过一定阈值的非空网格中选取代表点。但是当数据集中不同聚类的数据样本密度差异较大时,如网格大小不合理,部分低密度区域则有可能被排除在采样范围之外,造成代表点的缺失,并最终影响聚类结果。PBC 算法^[7]基于数据样本的 k 近邻关系衡量数据样本在数据集中心性,继而采样中心性高的数据样本作为骨干点,再将骨干点的影响力通过迭代传播分配给剩余数据样本,并根据分配到的影响力将剩余数据样本分到各个簇中。这种采样骨干点的方法能有效减少计算复杂度,但骨干点的选取对聚类结果影响较大。

2 CAPSULE 算法

任意形状聚类问题旨在将包含 N 个数据样本的数据集 $D = \{x_1, x_2, \dots, x_N\}$ 划分为 N_c 个数据样本聚类中，并输出各数据样本的聚类标签 $Label = \{l_1, l_2, \dots, l_N\}$, $l_i \in \{1, 2, \dots, N_c\}$, 其中 N_c 为用户指定的聚类个数, x_i 表示数据集中第 i 个数据样本, l_i 表示数据点 x_i 所属的聚类类别。值得指出的是, 在任意形状聚类问题中, 假设数据样本聚类的分布形状可能为非凸的、不规则的任意形状。因此, 传统的适用于识别分布形状为球形或凸形状的聚类算法并不适用于解决任意形状聚类问题。

本文提出一种利用形状保持采样的任意形状聚类算法 CAPSULE, 该算法包含以下三个阶段: (1) 形状保持采样, 从给定数据集 D 中采样 N_r 个数据样本作为代表点; (2) 代表点位置调整, 将各代表点从自身位置上逐渐向其 k 近邻代表点移动; (3) 基于类间最小距离的凝聚聚类, 将代表点采用凝聚聚类合并为 N_c 个聚类, 并将各代表点的类别标签作为其代表的数据集 D 中原始数据样本的类别标签。算法 1 列出了 CAPSULE 算法的伪代码。本节接下来的部分将对 CAPSULE 算法中的三个阶段给出具体介绍。

算法 1 CAPSULE 算法

输入: 数据集 D , 聚类个数 N_c , 代表点个数 N_r , k 近邻参数 k , 迭代次数 N_{iter}

输出: 数据集 D 中各数据样本的类别

1. 使用形状保持采样, 从数据集 D 的原始数据样本中获得 N_r 个采样点;
2. 以采样点为初始聚类中心, 运行 K 均值算法, 将最终聚类中心选为代表点; 各代表点受自身和 k 近邻代表点的共同作用力影响, 迭代调整位置, 直至迭代次数达到 N_{iter} ;
3. 计算类间最小距离, 对代表点进行凝聚聚类直至聚类个数降为 N_c , 将代表点的聚类标签赋给其代表的原始数据样本。

2.1 形状保持采样

形状保持采样包含两个步骤: (1) 去除数据集中离群点, (2) 在去除了离群点的数据集中不断采样, 通过采用让下一个采样点尽量远离已有采样点的策略, 获得在原始数据簇的分布区域内均匀连续分布的采样点。

形状保持采样首先剔除原始数据样本中离群点的原因在于: 离群点周围数据样本较少, 它所能代表的数据样本非常有限, 不能很好地支持采样的目的; 更重要的是, 离群点常位于聚类与聚类之间或者远离任何一个聚类。如果选到聚类之间的离群点作为代表点, 该代表点所代表的原始数据样本可能属于不同的聚类, 影响最终聚类结果的准确性; 如果选到远离任何一个聚类的离群点作为代表点, 则有可能与其他代表点都相距较远, 在后续的代表点聚类中成一个独立的 (但无意义的) 聚类。目前已存在许多相对成熟的离群点检测算法^[22-24], 本文选用 Bay 等人^[22]提出的快速离群点检测算法, 该算法可在接近线性的时间复杂度内完成离群点检测。

令 D' 表示去除了离群点的原始数据样本集合, $|D'|$ 表示该集合中数据样本个数, R_t 和 C_t 分别表示第 t 次采样后的采样点集合和候选点集合 (即 $C_t = D' \setminus R_t$)。 r_t 为第 t 个采样点, $d(p_i, p_j)$ 为 D' 中两个元素 p_i 与 p_j 的距离, p_i 的坐标为 $X_i = (x_{i,1}, x_{i,2}, \dots, x_{i,d})$ (d 为数据集维度), 第 $(t-1)$ 次采样后所有候选点与其距离最近的采样点的对应关系表示为:

$$M_t = \{p_i \rightarrow p_j | p_i \in C_t, p_j \in R_t, p_j = \underset{p_j \in R_t}{\operatorname{argmin}} d(p_i, p_j)\} \quad (1)$$

第 $(t-1)$ 次选取后所有候选点与其对应最近采样点的距离集合表示为:

$$DS_t = \{d(p_i, p_j) | p_i \rightarrow p_j \in M_t\} \quad (2)$$

在采样过程中 D' 中各数据样本只能属于候选点集合或者采样点集合, 即 $D' = R_t \cup C_t$ 且 $R_t \cap C_t = \emptyset$ 。

初始时, $R_0 = \emptyset$ 而 $C_0 = D'$ 每次采样从候选点集合中选一个数据样本, 将它移到采样点集合中。具体采样策略如下:

(1) 计算 D' 中数据样本分布中心坐标:

$$X_{mean} = (x_{mean,1}, x_{mean,2}, \dots, x_{mean,d}) \quad (3)$$

其中每个维度 j ($1 \leq j \leq d$) 取值为:

$$x_{mean,j} = \frac{1}{2} \left(\max_{1 \leq i \leq |D'|} x_{i,j} + \min_{1 \leq i \leq |D'|} x_{i,j} \right) \quad (4)$$

然后计算 D' 中各数据样本到中心坐标的距离, 取得距离中心坐标最近的数据样本作为第一个采样点;

(2) 当已经选取了 t 个采样点, 要选择第 $(t+1)$ 个采样点时: 对于每个候选元素 $p_i \in C_t$, 找到离其距离最近的采样点 $\underset{p_j \in R_t}{\operatorname{argmin}} d(p_i, p_j)$, 其中该候选点与其最近采样点间的距离可表示为 $d(p_i, \underset{p_j \in R_t}{\operatorname{argmin}} d(p_i, p_j))$, 从所有候选点中选取离

各自最近采样点距离最大的候选点作为第 $(t+1)$ 个采样点, 即

$$r_{t+1} = \underset{p_i \in C_t}{\operatorname{argmax}} d \left(p_i, \underset{p_j \in R_t}{\operatorname{argmin}} d(p_i, p_j) \right) \quad (5)$$

s. t. $1 \leq t \leq (N_r - 1)$

(3) 随着采样点集的逐渐扩展, 每次采样点会越来越接近已有采样点, 当经过连续若干次迭代过程后, 新增采样点与现有采样点集的最短距离趋于稳定, 在连续迭代过程中, 定义 $d_{min}(t)$ 表示当前第 t 次迭代中新增采样点到已有点集的最短距离, 计算前 5 次迭代过程中的最短距离平均值:

$$avg_{d_{min}} = \frac{d_{min}(t) + d_{min}(t-1) + d_{min}(t-2) + d_{min}(t-3) + d_{min}(t-4)}{5} \quad (6)$$

在连续迭代过程中, 如果多次满足:

$$\left| \frac{d_{min}(t) - avg_{d_{min}}}{avg_{d_{min}}} \right| < \varepsilon \quad (\varepsilon \text{ 为预设阈值}) \quad (7)$$

则认为新增采样点与现有采样点集的最短距离已趋于稳定, 这时采样点的分布已经达到了一定密度。这里要求多次满足的原因在于防止局部噪音影响对是否已达到“最短距离已趋

于稳定”这一收敛条件的判断。在本文实验过程中,如果连续5次迭代都小于阈值 ε ,则认为采样过程收敛,停止采样。

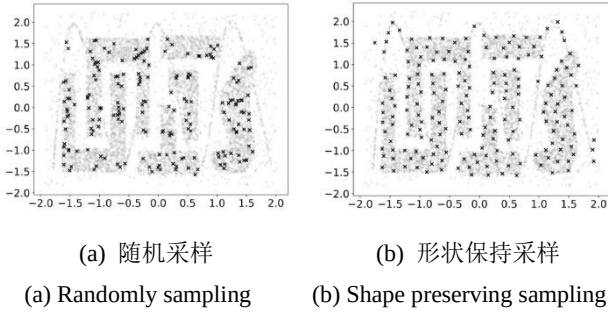


图1 采样效果对比

Fig.1 Comparison of sampling results

图1对比了随机采样获得的采样点与本文所提出的形状保持采样获得的采样点的分布情况(用黑色叉号表示)。可以看出,随机采样常造成采样点在某些区域过于密集而在某些区域过于稀疏,而形状保持采样所得的采样点则相对均匀连续地分布在原始数据簇中。

2.2 代表点选取与位置调整

采样完成后,本文首先将采样点作为初始聚类中心,使用K均值算法将 D' 中原始数据样本聚类。由于初始聚类中心相对均匀连续地分布在原始数据簇中,K均值算法可以快速将数据集分解成一系列大小相近连续分布的小聚类。本文使用各小聚类中离聚类中心最近的数据样本作为代表点,用于代表各小聚类中全部原始数据样本。这一步借鉴了基于聚类的采样方法的策略,其作用是将采样点的位置进行微调,使之移动到更接近局部密度分布中心的位置,从而增加最终所选取代表点的代表性。

为了使代表点能够更清晰地展示聚类结构,本文对代表点的位置进行调整使得同一类的代表点更加靠拢,而不同类的代表点之间的分界更加明显。为了达到这一目标,本文通过考察代表点的同质性,迭代调整代表点位置。为了考察代表点之间的同质性,基本思路是检查代表点在位置上是否足够接近,以及它们所代表的簇密度上是否具有连通性。更加具体地说,如果两个代表点满足以下三个条件,那么它们之间倾向于具有较高的同质性:

- (1) 两个代表点距离相近;
- (2) 代表点所代表的小聚类接触面大;
- (3) 代表点所代表的小聚类密度相近。

为定量地衡量代表点之间同质性,本文首先以所有代表点为顶点,构造k近邻图(各顶点与其k近邻之间加上无向边进行连通)。定义 L_i 和 L_j 两个小聚类, y_i 和 y_j 分别是各小聚类的代表点。为了定量衡量代表点之间的距离,构造高斯核函数定义距离度量 G_{ij} ,如式(8)所示:

$$G_{ij} = e^{-\frac{\|y_i - y_j\|^2}{ad_i + ad_j}} \quad (8)$$

其中, $ad_i = \frac{1}{k^2} \sum_{x_i \in kNN_{y_i}} \|x_i - y_i\|^2$,这里 kNN_{y_i} 表示代表点

y_i 在代表点集合中的k近邻, ad_i 表示 y_i 的平均近邻距离。 G_{ij} 越大表示代表点 y_i 和 y_j 之间距离相对其k近邻来说较近。

为了定量衡量代表点所代表的小聚类的接触面,定义小聚类 L_i 到 L_j 的接触面中数据样本集合为 L_{ij} :

$$L_{ij} = \{x \in L_i \mid \|x_i - y_j\| \leq \mu_{ij}\} \quad (9)$$

其中, μ_{ij} 表示代表点 y_i 和 y_j 间的距离, L_{ij} 表示 L_i 中到 y_j 的距离小于等于 μ_{ij} 的数据样本的集合。类似的, L_j 到 L_i 的接触面

数据样本集合为 L_{ji} 。如果 $\frac{|L_{ij}| + |L_{ji}|}{|L_i| + |L_j|}$ 值越高,则表示两个小聚类接触面中数据样本相对于两个小聚类来说所占比例越大,意味着两个小聚类具有相对较大的接触面。

为了定量衡量代表点所代表的小聚类密度的接近程度,定义小聚类 L_i 的局部密度为:

$$ld_i = \frac{|L_i|}{\sum_{x_t \in L_i} \|y_i - x_t\|} \quad (10)$$

其中,分子体现小聚类数据样本多少,分母体现小聚类分布范围大小。那么对于小聚类 L_i 和 L_j 而言,如果 $\frac{\min\{ld_i, ld_j\}}{\max\{ld_i, ld_j\}}$ 的值

越接近1,表示小聚类 L_i 和 L_j 的局部密度越接近。

综上所述,计算代表点同质性权重 W_{ij} 如式(11)所示:

$$W_{ij} = \begin{cases} \frac{\min\{ld_i, ld_j\}}{\max\{ld_i, ld_j\}} \cdot \frac{|L_{ij}| + |L_{ji}|}{|L_i| + |L_j|} \cdot G_{ij}, & y_i \text{和} y_j \text{连通} \\ 0, & \text{otherwise} \end{cases} \quad (11)$$

根据代表点之间的同质性权重,采用迭代位移的方式持续更新各个代表点的位置,使得同质性高的代表点离得更近。具体计算方法如式(12)所示:

$$y_i^{t+1} = \sum_j W_{ij}^* \cdot y_j^t \quad (12)$$

其中 t 表示迭代位移次数, y_i^t 为第 t 次位移后代表点 y_i 的位置, W^* 是经过归一化处理后的矩阵,其中元素为:

$$W_{ij}^* = \frac{R_{ij}}{\sum_l R_{il}}, \forall i, j \in \{1, 2, \dots, |S|\} \quad (13)$$

这里 S 表示代表点集合。值得指出的是,这种位移方法具有较好的收敛趋势,代表点位移量上限是随着位移迭代次数的增加而不断减小的。原因如下:

根据式(12)的位移方法有:

$$S^{(t)} = W^* \cdot S^{(t-1)}, \forall t \geq 1 \quad (14)$$

令位移矩阵 Z^t 表示第 t 轮迭代代表点的位移量,则

$$\begin{aligned} Z^{(t+1)} &= W^* S^{(t)} - S^{(t)} \\ &= W^* W^* S^{(t-1)} - W^* S^{(t-1)} \\ &= W^* (W^* S^{(t-1)} - S^{(t-1)}) = W^* Z^{(t)} \end{aligned}$$

即,

$$Z^{(t+1)} = \sum_{i=1}^{|S|} W_{ij}^* Z_j^{(t)} \quad (15)$$

其中 $Z_j^{(t)}$ 表示 $Z^{(t)}$ 矩阵的第 j 行。

设 $V^{(t)}$ 是 $Z_j^{(t)}, Z_j^{(t)}, \dots, Z_j^{(t)}$ 组成的最小凸区间（或凸包、凸多面体），则

$$V^{(t)} = \{ \{ \lambda_1 Z_1^{(t)} + \dots + \lambda_{|S|} Z_{|S|}^{(t)} | \lambda_i \geq 0, i = 1, 2, \dots, |S|, \sum_{i=1}^{|S|} \lambda_i = 1 \}$$

令 $|Z^{(t)}|_M$ 表示位移矩阵 $Z^{(t)}$ 的最大位移距离，则

$$|Z^{(t)}|_M = \max\{\|Z_1^{(t)}\|, \dots, \|Z_{|S|}^{(t)}\|\} \quad (16)$$

以下定理从理论上保证 $V^{(t)}$ 与 $|Z^{(t)}|_M$ 是随着位移次数 t 的增加呈递减趋势的。

定理 1: 当 $t=1,2,3,\dots$ 时，有 $V^{(t+1)} \subseteq V^t$ 和 $|Z^{(t+1)}|_M \leq |Z^{(t)}|_M$ 。如果所有 $i = 1, \dots, |S|$, $W_{ij}^* < 1$ ，则 $V^{(t+1)} \subset V^t$ 和 $|Z^{(t+1)}|_M < |Z^{(t)}|_M$ 。

证明：当 $i, 1 \leq i \leq |S|$ ，有 $Z_i^{(t+1)} = \sum_{j=1}^{|S|} W_{ij}^* Z_j^{(t)}$ 而且 $\sum_{j=1}^{|S|} W_{ij}^* = 1, W_{ij}^* \geq 0$ ，有 $Z_i^{(t+1)} \in V^t$ 和

$$\|Z_i^{(t+1)}\| = \left\| \sum_{j=1}^{|S|} W_{ij}^* Z_j^{(t)} \right\| \leq \left\| \sum_{j=1}^{|S|} W_{ij}^* |Z^{(t)}|_M \right\| = |Z^{(t)}|_M$$

即，对于任何 $1 \leq i \leq |S|$ ，有 $Z_i^{(t+1)} \in V^t$ 和 $\|Z_i^{(t+1)}\| \leq$

$|Z^{(t)}|_M$ ，然后 $V^{(t+1)} \subseteq V^t$ 和 $|Z^{(t+1)}|_M \leq |Z^{(t)}|_M$ 。

接下来，对于所有 $i = 1, \dots, |S|$, $W_{ii}^* < 1$ 。对于任何固定 $i, 1 \leq i \leq |S|$ ，因为 $W_{ii}^* < 1$ ，存在一些 $k \neq i, 1 \leq k \leq |S|, W_{ik}^* > 0$ ，然后

$$\begin{aligned} \|Z_i^{(t+1)}\| &= \left\| \sum_{j=1}^{|S|} W_{ij}^* Z_j^{(t)} \right\| \\ &\leq W_{ii}^* \|Z_i^{(t)}\| + W_{ik}^* \|Z_k^{(t)}\| \\ &\leq (W_{ii}^* + W_{ik}^*) \max\{\|Z_i^{(t)}\|, \|Z_k^{(t)}\|\} \end{aligned}$$

因为 $Z_i^{(t)} \neq Z_k^{(t)}$ ，上述两个不等式中的等号不能同时成立，因此

$$\begin{aligned} \|W_{ii}^* Z_i^{(t)} + W_{ik}^* Z_k^{(t)}\| &< (W_{ii}^* + W_{ik}^*) \max\{\|Z_i^{(t)}\|, \|Z_k^{(t)}\|\} \\ &\leq (W_{ii}^* + W_{ik}^*) |Z^{(t)}|_M \end{aligned}$$

因此得出

$$\begin{aligned} \|Z_i^{(t+1)}\| &\leq \|W_{ii}^* Z_i^{(t)} + W_{ik}^* Z_k^{(t)}\| + \sum_{j \neq i, k} W_{ij}^* |Z^{(t)}|_M \\ &< (W_{ii}^* + W_{ik}^*) |Z^{(t)}|_M + \sum_{j \neq i, k} W_{ij}^* |Z^{(t)}|_M = |Z^{(t)}|_M \end{aligned}$$

即， $|Z^{(t+1)}|_M < |Z^{(t)}|_M$ 。

因为 $V^{(t+1)} \subseteq V^t$ ，而且因为 $|Z^{(t+1)}|_M < |Z^{(t)}|_M$ ，得到 $V^{(t+1)} \neq V^t$ ，有 $V^{(t+1)} \subset V^t$ 。定理得证。

由以上定理可知， S 中代表点在位移过程中的存在凸包，导致位移大小具有上限，并随着迭代次数越来越小，造成代表点位移的收敛趋势。

值得指出的是，采用以上这种位移方式调整代表点位

置， k 值选择不宜过大。因为要保证代表点位移仅受其同质邻居的影响，而过大的 k 值会增加在不同聚类甚至离群点中寻找近邻关系的风险，影响位移效果及最后的聚类质量。

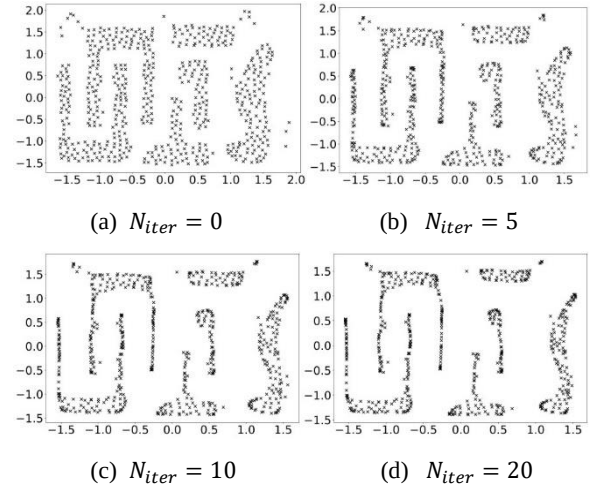


图2 代表点位置调整示例

Fig.2 Example of the position adjustment of representative data examples

图2直观地描绘了当代表点位置调整迭代次数 N_{iter} 逐渐增加时，代表点所反映的聚类结构。从图中可以看出，随着迭代次数的增加，代表点的位移量逐渐减小。并且在经过多次位置调整后，同一聚类中的代表点越来越靠近彼此，呈现出紧密而规则的形状；而不同聚类之间，则由于代表点相互远离而产生明显区分，这样就使得各个聚类更容易被识别和区分。

2.3 基于类间最小距离的凝聚聚类

完成代表点位置调整之后，拥有较高同质性的代表点会互相靠拢，因此，当两个代表点距离相近时，在原始样本中，它们更有可能属于相同的簇中。这种情况非常便于通过代表点之间的距离将同一任意形状聚类中各部分的代表点子集串联起来。因此，本文采用基于类间最小距离的凝聚聚类方法。该方法可以完成代表点的聚类操作。具体方法如下：初始时，将每个代表点视为一个独立的聚类；然后，迭代执行（1）计算每两个聚类之间的类间最小距离（2）将当前类间最小距离最小的两个聚类合并为一个新聚类这两步操作，直到聚类的个数减少至指定值。

完成代表点的凝聚聚类后，各代表点的类别标签将成为其所代表的原始数据样本的类别标签。对于在形状保持采样第一步中被去除的离群点来说，本文计算其到各类原始数据样本的距离，并将离其最近的原始数据样本的类别标签作为该离群点的类别标签。完成以上步骤后，本文便完成了对给定数据集全部原始数据样本的聚类。

2.4 算法复杂度分析

CAPSULE 算法的第一阶段为形状保持采样，所采用的离群点检测方法^[22]时间复杂度接近线性，约 $O(N)$ ，其中 N 表示给定数据集中全部原始数据样本的数量；通过均匀连续采

样获得初始聚类中心点花费时间为 $O(N_r \cdot N)$ ，其中 N_r 为所获得初始聚类中心的个数（也就是最终代表点个数）；调用K均值算法获得最终的代表点集合花费时间为 $O(N_r \cdot N)$ 。CAPSULE算法的第二阶段为代表点位置调整，确定代表点间k近邻关系需要耗时 $O(N_r^2)$ ；迭代地调整代表点位置需要耗时 $O(N_{iter} \cdot N_r \cdot k)$ ，其中 N_{iter} 为迭代次数（ $N_{iter} \ll N_r, k \ll N_r$ ）。CAPSULE算法的第三阶段为基于类间最小距离的凝聚聚类，通常的凝聚聚类具有三次方的时间复杂度，不过本文可以通过借助堆存储最近邻的距离，使得凝聚聚类的时间复杂度降至 $O(N_r^2 \log N_r)$ 。综上，本文所提的CAPSULE算法的时间复杂度为 $O(N_r \cdot N + N_r^2 \log N_r)$ 。

3 实验分析

3.1 实验设置

实验数据集包括真实数据集和人工数据集。其中，真实数据集为5个来自UCI机器学习仓库^[25]的数据集。表1列出了这些UCI数据集的相关信息。人工数据集包含两个二维数据集（DS2.1、DS2.2）和一个三维数据集（DS2.3），常用于验证任意形状聚类算法的有效性^[3-6]。

此外，为评估算法的可扩展性，本文还利用数据生成工具SynDECA^[26]生成了两组数据集DS3.1~DS3.7和DS4.1~DS4.7。第一组数据集DS3.1~DS3.7为7个8维数据集，数据样本个数为5000至35000（间隔为5000）；第二组数据集DS4.1~DS4.7中每个数据集包含12000个数据样本，维度为8至56（间隔是8）。

表1 UCI数据集

Table 1 UCI datasets

数据集	数据集名称	样本个数	维度	类别
DS1.1	Image Segmentation	2310	19	7
DS1.2	Landsat Satellite	6435	36	6
DS1.3	Dry Bean Dataset	13611	16	7
DS1.4	Pen Digits	10992	16	10
DS1.5	Letter Recognition	20000	16	26

本文全部实验运行于装配有16G内存和3.3GHz处理器、运行Win10操作系统台式电脑上。全部算法均采用Python实现。算法聚类结果的准确性用调整兰德系数（Adjusted Rand Index, ARI）、调整互信息（Adjusted Mutual Information, AMI）、归一化互信息（Normalized Mutual Information, NMI）衡量，此外在真实数据集中，还会参考真实标签使用准确率ACC（Accuracy）进行衡量。

当样本集 D 包含 m 个类时，聚类准确率的计算公式如式（17）所示：

$$Acc = \frac{1}{|D|} \sum_{i=1}^m a_i \quad (17)$$

a_i 表示被正确划分到第 i 个类中的样本数； $|D|$ 为样本总数。

ARI基于兰德指数（Rand Index）改进而来，用于衡量两个聚类分配之间的一致性。ARI考虑了随机聚类的期望值，

从而提供了更合理的聚类性能评价。兰德指数的计算公式如式（18）所示：

$$RI = \frac{a+b}{a+b+c+d} \quad (18)$$

其中， a 是指在两个聚类结果中都被正确分配到同一个簇的样本对的数量； b 是指在两个聚类结果中都被正确分配到不同簇的样本对的数量； c 是指在第一个聚类中同簇，但在第二个聚类中不同簇的样本对的数量； d 是指在第一个聚类中不同簇，但在第二个聚类中同簇的样本对的数量。RI的取值范围在[0,1]区间范围内，为了使其值在随机聚类情况下接近0，而在完美聚类情况下接近1，进一步考虑ARI：

$$ARI = \frac{RI - E(RI)}{\max(RI) - E(RI)} \quad (19)$$

ARI在[-1,1]范围内取值，ARI取值越接近1，表明算法的聚类质量越好。

AMI和NMI都是基于互信息来评估聚类结果与真实标签的一致性。互信息衡量两个变量之间的相互依赖关系。互信息的计算公式如式（20）所示：

$$MI(U, V) = \sum_{i=1}^K \sum_{j=1}^L P(U_i, V_j) \log \frac{P(U_i, V_j)}{P(U_i)P(V_j)} \quad (20)$$

其中， U 和 V 分别表示聚类结果和真实标签的分布， $P(U_i)$ 是数据样本属于聚类 U_i 的概率， $P(V_j)$ 是数据样本属于聚类 V_j 的概率， $P(U_i, V_j)$ 是数据样本同时属于聚类 U_i 和 V_j 的联合概率。

AMI考虑了在随机分配标签的情况下可能出现的互信息，减小了由于标签数量或分布差异导致的偏差：

$$AMI = \frac{MI(U, V) - E[MI(U, V)]}{\max(H(U), H(V)) - E[MI(U, V)]} \quad (21)$$

其中， $H(U)$ 和 $H(V)$ 分别表示聚类结果和真实标签的熵； $E[MI]$ 为随机情况下的期望互信息。AMI的取值范围为[0,1]，其中1表示完全一致，0表示和随机分配标签无区别。

NMI对互信息进行了规范化，使互信息的取值范围在0到1之间，越接近1表示聚类结果与真实标签的匹配度越高，计算公式如式（22）所示：

$$NMI(U, V) = \frac{2 * MI(U, V)}{H(U) + H(V)} \quad (22)$$

3.2 参数选择

代表点个数 N_r ：为选取代表点，CAPSULE算法会先自动选择一定数量的采样点作为K均值算法的初始聚类中心来分解数据集以获得代表点。采样点个数 N_r 即代表点个数。 N_r 大小受式（7）中采样停止阈值 ε 的影响，本文在数据集DS1.1~DS1.5上选取不同的采样停止阈值进行实验，结果如图3（a）所示。

从图3（a）可以看出，随着 ε 的取值的减小，算法准确性呈上升趋势，但当 ε 的取值大于等于0.0005时，算法准确性几乎达到最高值，并趋于稳定。但注意继续减小 ε 会使得 N_r

迅速增大,会增加算法要处理的数据量。因此,在本文后续实验中,取 $\varepsilon=0.0005$ 作为采样停止条件。表 2 报告使用该采样停止条件时,数据集 DS1.1~DS1.5 的收缩比例(即 N_r/N)。

表 2 数据集收缩比例

Table 2 Shrinking ratios of datasets

数据集	数据集样本个数 N	代表点个数 N_r	收缩比例
DS1.1	2310	475	0.206
DS1.2	6435	227	0.035
DS1.3	13611	258	0.019
DS1.4	10992	180	0.016
DS1.5	20000	781	0.039

本文还通过按比例调整不同的代表点个数进行实验,观察代表点个数变化对 CAPSULE 算法聚类结果的影响。实验结果如图 3 (b) 所示: 本文方法选取的代表点个数可帮助 CAPSULE 算法聚类结果的准确性达到或者接近最优。

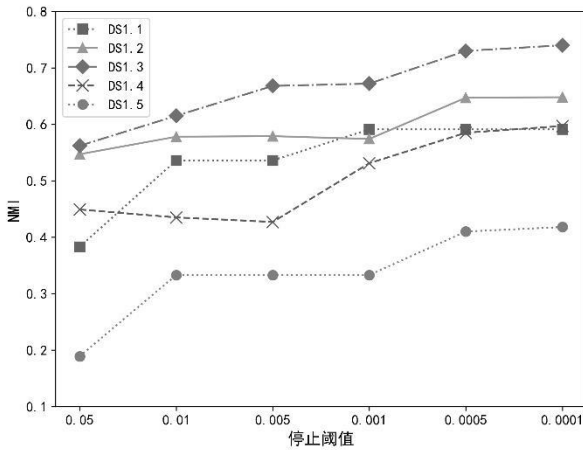
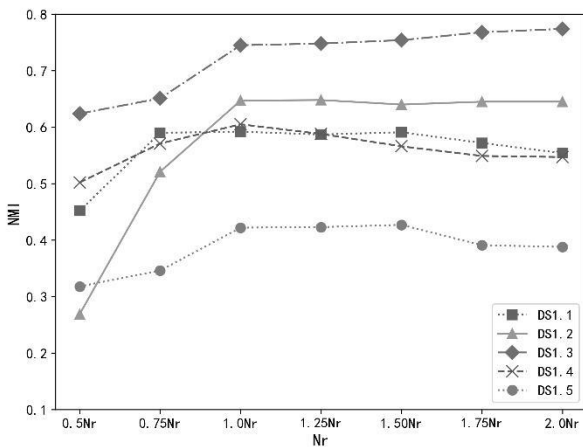
(a) ε 对 CAPSULE 算法聚类结果的影响(a) Effect of ε on CAPSULE(b) N_r 值对 CAPSULE 算法聚类结果的影响(b) Effect of N_r on CAPSULE

图 3 代表点个数对 CAPSULE 算法聚类结果的影响

Fig.3 Effect of the number of representative points on CAPSULE

此外,为评估本文提出的代表点选取方法对 CAPSULE

算法聚类结果的影响,本文在 DS1.1~DS1.5 上采用随机采样方法替换本文提出的代表点选取方法。表 3 给出了准确率对比结果,其中,采用随机采样的 CAPSULE 算法报告了其执行 20 次时的平均 NMI 及其标准差。实验结果表明相比于随机采样,采用本文所提出的代表点选取方法能有效帮助 CAPSULE 算法提高聚类结果质量,获得更高的准确率。

表 3 采样方法对 CAPSULE 算法聚类结果的影响

Table 3 Effect of Sampling Methods on CAPSULE

数据集	CAPSULE	使用随机种子的CAPSULE
DS1.1	0.592	0.481±0.056
DS1.2	0.647	0.568±0.056
DS1.3	0.745	0.699±0.014
DS1.4	0.605	0.548±0.032
DS1.5	0.422	0.390±0.025

k 值: 本文方法进行代表点位置调整时,需用到代表点的 k 近邻关系。为观察 k 值对本文方法聚类效果的影响,本文在数据集 DS1.1~DS1.5 使用不同 k 值进行实验。实验结果如图 4 所示,从中可以观察到: 在各数据集上,随着 k 值不断增大, CAPSULE 算法聚类结果准确性都有一个先上升再下降的趋势; 当 k 值取 4 时,由于 k 值过小,不能完全起到利用周边同质近邻的作用,这时最终聚类效果并未达到最优; 当 k 值取 6~8 时,聚类效果相对最好; 但随着 k 值继续增大,聚类效果出现下降趋势,这是因为较大的 k 值增加了在不同聚类甚至离群点中寻找近邻关系的风险,影响位移效果及最后的聚类质量。实验结果也验证了在本文 2.2 节提出的 k 值不宜过大的观点。因此,在本文后续实验中,将 k 值设为 6。

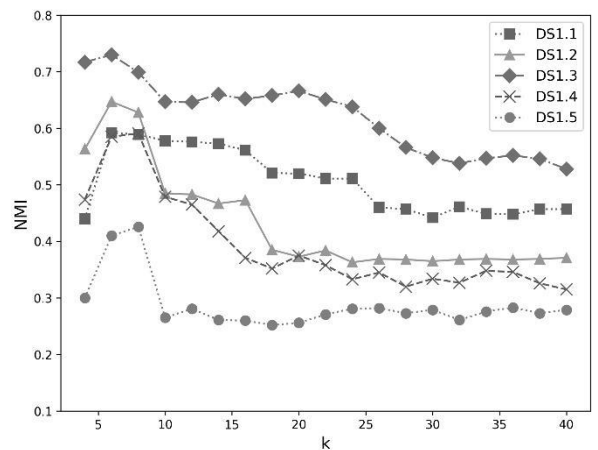


图 4 k 值对 CAPSULE 算法聚类结果的影响

Fig.4 Effect of k on CAPSULE

迭代次数 N_{iter} : 本文方法对代表点要进行 N_{iter} 轮迭代式的位置调整。为观察迭代调整次数 N_{iter} 对 CAPSULE 算法最终聚类效果的影响,本文在数据集 DS1.1~DS1.5 上使用不同 N_{iter} 值来运行 CAPSULE 算法。图 5 报告了迭代调整次数 N_{iter} 变化时 CAPSULE 算法聚类结果的准确性,从中可以观察到: 在各数据集上,随着迭代调整次数 N_{iter} 不断增大,

CAPSULE 算法聚类结果准确性都有一个先上升再逐渐平稳的趋势；通常，超过 20 次迭代调整后，CAPSULE 算法聚类结果的准确性在各数据集上均趋于稳定。因此，在本文后续实验中，将代表点位置的迭代调整次数 N_{iter} 设为 20。

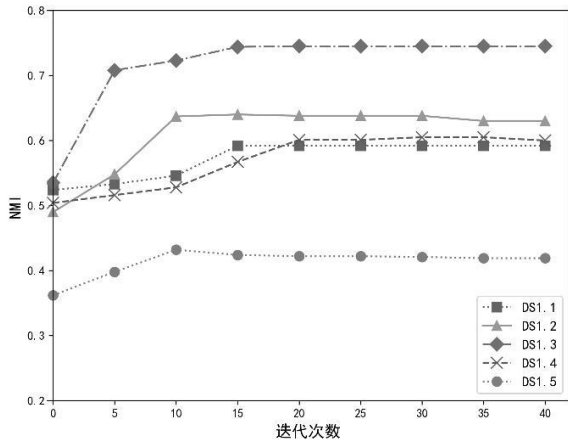
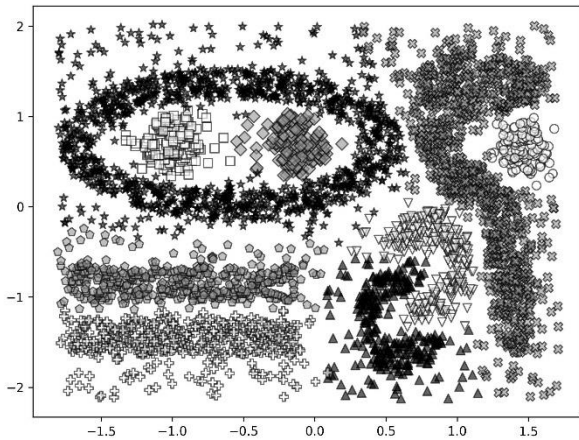


图 5 迭代调整次数 N_{iter} 对 CAPSULE 算法聚类结果的影响

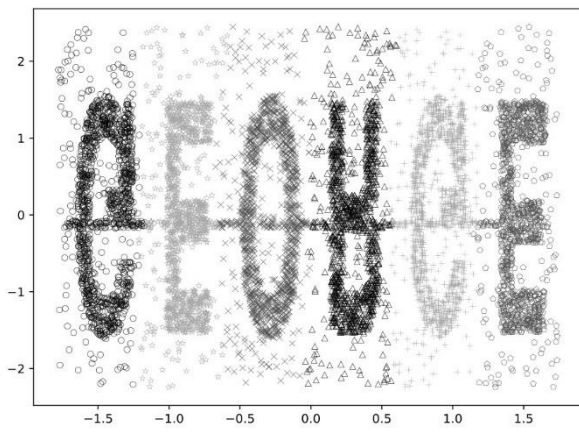
Fig.5 Effect of N_{iter} on CAPSULE

3.3 聚类能力验证

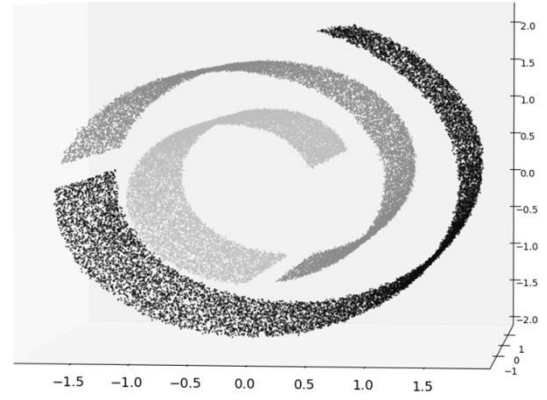
为验证 CAPSULE 算法具有识别任意形状聚类的能力，图 6 给出了 CAPSULE 算法在 3 个常用的人工数据集上的聚类结果（不同标记表示不同聚类）。结果表明 CAPSULE 算法能够准确的分析识别出其中各个任意形状数据集簇。



(a) DS2.1



(b) DS2.2



(c) DS2.3

图 6 CAPSULE 算法在人工数据集上的聚类结果

Fig.6 Clustering results of CAPSULE on synthetic datasets

3.4 算法性能对比

本文将 CAPSULE 算法与基于图的方法 Chameleon^[8]和 MDSC^[12]、基于密度的方法 DBSCAN^[14]、EC^[19]和 VDPC^[21]（对于基于密度方法所判断出的离群点，在本文中选择在聚类完成后，将其聚类至最近的非离群点所属的簇）、基于采样的方法 CLASP^[6]、IK-means^[3]和 PBC^[7]进行比较。由于基于图的方法和基于密度的方法对参数较为敏感，本文参考 MDSC 算法^[12]的实验设置中对以上两类方法建议的参数取值范围，在取值范围内均匀选取参数进行实验，对比不同参数（或参数组合）的效果，选取其中准确性最高的聚类结果作为基于图的方法和基于密度的方法的最终聚类结果。对于基于采样的方法，使其采样比例等于或最大程度地接近本文 CAPSULE 算法的采样比例（即表 2 中所示的收缩比例）。

表 4 列出了各算法在 UCI 数据集上的运行时间，可观察到：相较于对比算法，CAPSULE 算法所需运行时间明显更少，且在越大的数据集上，这种运行效率上的优势越明显。

表 4 在 UCI 数据集上的运行时间对比(秒)

Table 4 Runtime comparison on UCI datasets (seconds)

算法/数据集	DS1.1	DS1.2	DS1.3	DS1.4	DS1.5
Chameleon	0.227	0.755	1.611	47.918	1102.40
MDSC	0.148	1.353	9.876	155.61	1295.72
DBSCAN	0.132	0.924	1.215	1.342	14.508
EC	0.104	0.627	2.649	6.085	21.864
VDPC	0.108	0.562	2.120	9.092	64.148
CLASP	0.102	0.572	1.340	4.825	57.940
IK-means	0.196	0.784	1.685	2.207	11.194
PBC	0.125	0.550	1.182	1.298	6.457
CAPSULE	0.101	0.475	1.107	0.601	3.985

表 5 列出了各算法在 UCI 真实数据集上的聚类结果准确性，可观察到：相较于对比算法，CAPSULE 算法聚类结果准确性相对更高，并且在数据样本更多、聚类个数更多的数据集上，CAPSULE 算法体现出更明显的准确性优势。

表 5 聚类算法性能比较

Table 5 Performance comparison of clustering algorithms

数据集		算法								
		Chameleon	MDSC	DBSCAN	EC	VDPC	CLASP	IK-means	PBC	CAPSULE
DS1.1	Acc	0.336	0.516	0.421	0.585	0.592	0.488	0.548	0.552	0.604
	ARI	0.121	0.384	0.231	0.401	0.388	0.375	0.423	0.410	0.454
	AMI	0.358	0.536	0.502	0.530	0.466	0.563	0.580	0.582	0.589
	NMI	0.360	0.537	0.504	0.532	0.466	0.564	0.581	0.584	0.592
DS1.2	Acc	0.356	0.607	0.520	0.618	0.588	0.620	0.664	0.699	0.736
	ARI	0.194	0.491	0.428	0.379	0.341	0.420	0.572	0.584	0.606
	AMI	0.268	0.625	0.545	0.540	0.543	0.487	0.608	0.630	0.647
	NMI	0.270	0.626	0.546	0.541	0.544	0.488	0.608	0.631	0.647
DS1.3	Acc	0.603	0.553	0.398	0.592	0.608	0.706	0.720	0.639	0.757
	ARI	0.440	0.340	0.133	0.405	0.388	0.410	0.582	0.410	0.617
	AMI	0.531	0.509	0.263	0.517	0.542	0.596	0.607	0.558	0.744
	NMI	0.531	0.510	0.264	0.517	0.544	0.597	0.608	0.558	0.745
DS1.4	Acc	0.312	0.298	0.425	0.481	0.588	0.601	0.620	0.594	0.627
	ARI	0.201	0.119	0.058	0.368	0.388	0.371	0.352	0.381	0.394
	AMI	0.419	0.348	0.216	0.549	0.601	0.573	0.592	0.582	0.603
	NMI	0.421	0.349	0.217	0.550	0.602	0.575	0.592	0.584	0.605
DS1.5	Acc	0.250	0.177	0.181	0.257	0.311	0.306	0.324	0.294	0.357
	ARI	0.130	0.023	0.214	0.059	0.204	0.175	0.218	0.182	0.301
	AMI	0.359	0.310	0.252	0.253	0.388	0.411	0.399	0.400	0.421
	NMI	0.362	0.315	0.255	0.257	0.390	0.412	0.402	0.402	0.422

3.5 算法可扩展性对比

为了评估数据样本数量对算法运行时间的影响，本文在 7 个数据集 DS3.1~DS3.7（各数据集维度一致，数据样本数量变化范围为 5000 至 35000）上运行 CAPSULE 算法和对比算法。表 6 给出了运行时间对比结果，从中可以观察到：随着数据样本数量增加，各算法的运行时间皆呈上升趋势；在数据样本数量较小时，CAPSULE 算法的运行时间轻微多于 PBC 算法的运行时间，但相差不大；随着数据样本数量

超过 10000 后，CAPSULE 算法的运行时间始终少于各对比算法，且数据样本越多，这种运行时间上的优势越明显。

为了评估数据集维度对算法运行时间的影响，本文在 7 个数据集 DS4.1~DS4.7（各数据集数据样本数量一致，维度变化范围为 8 至 56）上运行 CAPSULE 算法和现有代表性方法。表 7 给出了运行时间对比结果，从中可以观察到：随着数据集维度的增加，各算法的运行时间都呈上升趋势，但 CAPSULE 运行时间的上升速率低于现有代表性方法。

表 6 数据样本数量的可扩展性研究（秒）

Table 6 Scalability to number of data examples (seconds)

算法/数据集（样本数量）	DS3.1(5000)	DS3.2(10000)	DS3.3(15000)	DS3.4(20000)	DS3.5(25000)	DS3.6(30000)	DS3.7(35000)
CHAMELEON	0.365	1.939	8.584	14.301	26.939	32.608	38.458
MDSC	0.720	3.922	11.663	29.000	68.548	123.039	188.389
DBSCAN	16.465	37.254	40.496	47.097	58.267	79.855	87.597
EC	2.215	8.730	19.509	35.659	56.418	162.486	262.891
VDPC	2.306	9.214	17.426	40.268	55.240	140.924	220.612
CLASP	0.362	1.920	2.421	6.025	12.520	19.344	28.627
IK-means	0.328	0.918	2.031	4.820	8.998	12.876	17.024
PBC	0.205	0.914	1.990	4.020	7.450	9.694	14.554
CAPSULE	0.213	0.916	1.962	3.277	6.393	7.733	10.379

表 7 数据样本维度的可扩展性研究(秒)

Table 7 Scalability to number of data examples (seconds)

算法/数据集(维度)	DS4.1(8)	DS4.2(16)	DS4.3(24)	DS4.4(32)	DS4.5(40)	DS4.6(48)	DS4.7(56)
CHAMELEON	1.223	1.654	1.847	2.075	2.386	2.550	2.739
MDSC	6.332	7.072	7.108	7.158	7.203	7.248	7.651
DBSCAN	2.555	3.617	5.115	7.731	13.018	15.944	20.665
EC	12.244	12.456	12.684	12.824	12.945	13.171	13.306
VDPC	11.298	12.002	12.145	12.201	12.354	12.871	13.042
CLASP	1.208	1.429	1.620	1.871	2.011	2.200	2.541
IK-means	1.180	1.210	1.352	1.380	1.402	1.458	1.502
PBC	1.020	1.148	1.220	1.285	1.347	1.401	1.446
CAPSULE	0.999	1.024	1.040	1.045	1.063	1.079	1.161

3.6 算法局限性

尽管 CAPSULE 算法在实验所用的中低维度的数据集上表现较好,但仍存在以下局限性:(1) CAPSULE 算法需要用户指定聚类个数,不能自动确定聚类个数;(2)由于 CAPSULE 算法基于欧式距离来衡量数据样本之间的相似程度,当数据维度增加时,数据样本之间的欧式距离差异会趋于变小,使得数据样本之间的相似程度变得模糊,增加 CAPSULE 算法在高维数据集上聚类结果准确性下降的风险。

4 结论

本文提出了一种基于形状保持采样的任意形状聚类方法。该方法首先在原始数据样本分布区域内进行均匀连续采样,使所得采样点保持原始数据簇的分布形状;然后,以采样点为初始聚类中心,运行 K 均值聚类算法,对原始数据簇进行凸分解,以得到最终聚类中心为代表点,并调整各代表点的位置使之与其近邻的同质代表点逐渐靠拢;最后,用类间最小距离对代表点进行凝聚聚类,从而将处在同一任意形状聚类中各部分的代表点逐渐合并起来,完成任意形状聚类。在人工与真实数据集上的实验结果验证了本文方法处理任意形状聚类的能力和运行效率。

References:

- [1] Ran X, Suyaraj N, Tepsan W, et al. A hybrid genetic-fuzzy ant colony optimization algorithm for automatic K-means clustering in urban global positioning system[J]. Engineering Applications of Artificial Intelligence, 2024, 137: 109237.
- [2] Guha S, Rastogi R, Shim K. CURE: An efficient clustering algorithm for large databases[J]. ACM SIGMOD Record, 1998, 27(2): 73-84.
- [3] He H, He Y, Wang F, et al. Improved K-means algorithm for clustering non-spherical data[J]. Expert Systems, 2022, 39(9): e13062.
- [4] Chaoji V, Al Hasan M, Salem S, et al. SPARCL: An effective and efficient algorithm for mining arbitrary shape-based clusters[J]. Knowledge and Information Systems, 2009, 21(2): 201-229.
- [5] Chaoji V, Li G, Yildirim H, et al. ABACUS: Mining arbitrary shaped clusters from large datasets based on backbone identification[C]//SIAM International Conference on Data Mining, 2011: 295-306.
- [6] Huang H, Gao Y, Chiew K, et al. Towards effective and efficient mining of arbitrary shaped clusters[C]//IEEE International Conference on Data Engineering, 2014: 28-39.
- [7] Motallebi H. Efficient and robust clustering based on backbone identification[J]. Pattern Recognition, 2024, 155: 110635.
- [8] Karypis G, Han E H, Kumar V. Chameleon: Hierarchical clustering using dynamic modeling[J]. Computer, 1999, 32(8): 68-75.
- [9] Ertöz L, Steinbach M, Kumar V. Finding clusters of different sizes, shapes, and densities in noisy, high dimensional data[C]//SIAM International Conference on Data Mining, 2003: 47-58.
- [10] Qin Y, Yu Z L, Wang C D, et al. A novel clustering method based on hybrid k-nearest-neighbor graph[J]. Pattern Recognition, 2018, 74: 1-14.
- [11] Von Luxburg U. A tutorial on spectral clustering[J]. Statistics and Computing, 2007, 17: 395-416.
- [12] Yue G L, Deng A S, Qu Y P, et al. Stratified multi-density spectral clustering using Gaussian mixture model[J]. Information Sciences, 2023, 633: 182-203.
- [13] Wang Y, Qian J, Hassan M, et al. Density peak clustering algorithms: A review on the decade 2014-2023[J]. Expert Systems with Applications, 2024, 238: 121860.
- [14] Ester M, Kriegel H P, Sander J, et al. A density-based algorithm for discovering clusters in large spatial databases with noise[C]//International Conference on Knowledge Discovery and Data Mining, 1996: 226-231.
- [15] Ankerst M, Breunig M M, Kriegel H P, et al. OPTICS: Ordering points to identify the clustering structure[J]. ACM SIGMOD Record, 1999, 28(2): 49-60.
- [16] Hinneburg A, Keim D A. An efficient approach to clustering in large multimedia databases with noise[C]//International Conference on Knowledge Discovery and Data Mining, 1998: 58-65.

- [17] Rodriguez A, Laio A. Clustering by fast search and find of density peaks[J]. Science, 2014, 344(6191): 1492-1496.
- [18] Chen M, Li L, Wang B, et al. Effectively clustering by finding density backbone based-on kNN[J]. Pattern Recognition, 2016, 60: 486-498.
- [19] Wang S, Li Q, Zhao C, et al. Extreme clustering—a clustering method via density extreme points[J]. Information Sciences, 2021, 542: 24-39.
- [20] Zhang X Y, Yun W G. Sharing K-nearest neighbors and multiple assignment policies density peaks clustering algorithm [J]. Journal of Chinese Computer Systems, 2023, 44(1): 75-82.
- [21] Wang Y, Wang D, Zhou Y, et al. VDPC: Variational density peak clustering algorithm[J]. Information Sciences, 2023, 621: 627-651.
- [22] Bay S D, Schwabacher M. Mining distance-based outliers in near linear time with randomization and a simple pruning rule[C]//ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2003: 29-38.
- [23] Angiulli F, Pizzuti C. Fast outlier detection in high dimensional spaces[C]//European Conference on Principles of Data Mining and knowledge Discovery, 2002: 15-27.
- [24] Breunig M M, Kriegel H P, Ng R T, et al. LOF: Identifying density-based local outliers[C]//ACM SIGMOD International Conference on Management of Data, 2000: 93-104.
- [25] Dua D, Graff C. UCI machine learning repository. 2019. <http://archive.ics.uci.edu/ml>
- [26] Vennam J R, Vadapalli S. SynDECA: A tool to generate synthetic datasets for evaluation of clustering algorithms[C]//International Conference on Management of Data, 2005: 27-36.

附中文参考文献：

- [20] 张新元, 负卫国. 共享 K 近邻和多分配策略的密度峰值聚类算法[J]. 小型微型计算机系统, 2023, 44(1): 75-82.